NANYANG TECHNOLOGICAL UNIVERSITY SINGAPORE

SC3000 Artificial Intelligence Assignment 2: Learning to Use Prolog as a Logic Programming Tool

Lingyi LIU	**Junhao KE**	**Junwei Daniel LAW**
U2340111G	U2423264H	U2423201C

Team NanoGangstas, Tutorial group TEL2

Abstract

This report writes two problems as Prolog knowledge bases and shows the proofs as SWI-Prolog traces. In Exercise 1, we translate a research data policy into first-order logic and Prolog, and prove that anna violates it. In Exercise 2, we define a recursive rule for software dependencies, compare two orders of its clauses, and explain why a cyclic dependency can make the search run forever. All traces are real output from an interactive SWI-Prolog 10.0.2 session: `trace.` before the query, then `<enter>` (creep) at each step.

Member	Contribution
Lingyi LIU	Attempted both exercises, compared and evaluated answers with the team, and prepared the final report and traces
Junhao KE	Attempted both exercises, compared and evaluated answers with the team, and iterated on and reviewed the final report
Junwei Daniel LAW	Attempted both exercises, compared and evaluated answers with the team, and iterated on and reviewed the final report

1 Research Data Sharing (Exercise 1)

1.1 First-order logic

Table 1 translates each statement. Intern(x, g) reads "x is an intern at research group g", and Authorized(x, d) reads "x is authorized to publish d".

Table 1: First-order logic for each statement of Exercise 1.

Statement	First-order logic
A research group called visionlab owns a dataset called medset.	ResearchGroup(visionlab) $\wedge$ Dataset(medset) $\wedge$ Owns(visionlab, medset)
The dataset contains patient data.	ContainsPatientData(medset)
anna is an intern at visionlab.	Intern(anna, visionlab)
anna uploaded medset to a public repository.	UploadedToPublicRepo(anna, medset)
Any dataset containing patient data is considered sensitive.	$\forall d$ [Dataset(d) $\wedge$ ContainsPatientData(d) $\rightarrow$ Sensitive(d)]
An intern is not authorized to publish sensitive datasets owned by their research group.	$\forall x\ \forall g\ \forall d$ [Intern(x, g) $\wedge$ Owns(g, d) $\wedge$ Sensitive(d) $\rightarrow$ $\neg$Authorized(x, d)]
Uploading a dataset to a public repository is considered publishing the dataset.	$\forall x\ \forall d$ [UploadedToPublicRepo(x, d) $\rightarrow$ Publishes(x, d)]
Anyone who publishes a sensitive dataset without authorization violates the research data policy.	$\forall x\ \forall d$ [Publishes(x, d) $\wedge$ Sensitive(d) $\wedge$ $\neg$Authorized(x, d) $\rightarrow$ ViolatesPolicy(x)]

1.2 Prolog clauses

A Prolog rule (a Horn clause) cannot have a negative conclusion, so we cannot write $\neg$Authorized(x, d) as the head of a rule. Instead, we add an ordinary predicate, `not_authorized(X, D)`, which means "X is not authorized to publish D".

Notably, we did not write `\+ authorized(X, D)` (negation as failure), because `\+` only means "this cannot be found in the knowledge base". We have no `authorized` facts, so the check would pass for everyone and the intern rule would never be used. A professor who uploaded a sensitive dataset would also count as violating the policy, even though the statements only say that interns are not authorized.

Figure 1 shows the clauses.

```
% Exercise 1.2: Research Data Sharing
% Query: violates_policy(anna).

research_group(visionlab).
dataset(medset).
owns(visionlab, medset).
contains_patient_data(medset).
intern(anna, visionlab).
uploaded_to_public_repo(anna, medset).

sensitive(D) :- dataset(D), contains_patient_data(D).
not_authorized(X, D) :- intern(X, G), owns(G, D), sensitive(D).
publishes(X, D) :- uploaded_to_public_repo(X, D).
violates_policy(X) :- publishes(X, D), sensitive(D), not_authorized(X, D).
```

Figure 1: Knowledge base for Exercise 1 (`NanoGangstas_qn_1_2.pl`).

1.3 Proof that anna violates the research data policy

The query `violates_policy(anna)` matches the head of the `violates_policy` rule, which leaves three subgoals:

1. `publishes(anna, D)`: the publishing rule and the upload fact give **D = medset**.
2. `sensitive(medset)`: the sensitivity rule and the facts `dataset(medset)` and `contains_patient_data(medset)` prove it.
3. `not_authorized(anna, medset)`: the intern rule needs three more things. `intern(anna, G)` gives G = visionlab, `owns(visionlab, medset)` is a fact, and `sensitive(medset)` is proved again, as in step 2.

All three subgoals are proved, so `violates_policy(anna)` is **true**. Figure 2 shows the trace.

```
?- trace.
true.

[trace] ?- violates_policy(anna).
   Call: (12) violates_policy(anna) ? creep
   Call: (13) publishes(anna, _14444) ? creep
   Call: (14) uploaded_to_public_repo(anna, _14444) ? creep
   Exit: (14) uploaded_to_public_repo(anna, medset) ? creep
   Exit: (13) publishes(anna, medset) ? creep
   Call: (13) sensitive(medset) ? creep
   Call: (14) dataset(medset) ? creep
   Exit: (14) dataset(medset) ? creep
   Call: (14) contains_patient_data(medset) ? creep
   Exit: (14) contains_patient_data(medset) ? creep
   Exit: (13) sensitive(medset) ? creep
   Call: (13) not_authorized(anna, medset) ? creep
   Call: (14) intern(anna, _24168) ? creep
   Exit: (14) intern(anna, visionlab) ? creep
   Call: (14) owns(visionlab, medset) ? creep
   Exit: (14) owns(visionlab, medset) ? creep
   Call: (14) sensitive(medset) ? creep
   Call: (15) dataset(medset) ? creep
   Exit: (15) dataset(medset) ? creep
   Call: (15) contains_patient_data(medset) ? creep
   Exit: (15) contains_patient_data(medset) ? creep
   Exit: (14) sensitive(medset) ? creep
   Exit: (13) not_authorized(anna, medset) ? creep
   Exit: (12) violates_policy(anna) ? creep
true.
```

Figure 2: Trace of the query `violates_policy(anna)`.

2 Software Dependency Reasoning (Exercise 2)

2.1 Knowledge base and recursive rule

Figure 3 shows the facts and the rule. The first clause is the base case (a direct dependency). The second clause is the recursive case: X depends on some Z, and Z requires Y.

```
% Exercise 2.1: Software Dependency Reasoning
% Queries: requires(visionapp, P).        % press ; to list every package
%          requires(visionapp, driver).

depends_on(visionapp, pytorch).
depends_on(visionapp, opencv).
depends_on(pytorch, cuda).
depends_on(opencv, numpy).
depends_on(cuda, driver).

requires(X, Y) :- depends_on(X, Y).
requires(X, Y) :- depends_on(X, Z), requires(Z, Y).
```

Figure 3: Knowledge base for Exercise 2.1 (`NanoGangstas_qn_2_1.pl`).

Pressing `;` returns every package that visionapp requires:

```
?- requires(visionapp, P).
P = pytorch ;
P = opencv ;
P = cuda ;
P = driver ;
P = numpy ;
false.
```

So **visionapp requires pytorch, opencv, cuda, driver and numpy**.

Figure 4 shows the trace for `requires(visionapp, driver)`. The package driver is not a direct dependency of visionapp, so the first clause fails and Prolog backtracks into the recursive clause. That clause follows visionapp → pytorch, where the direct check fails again, and then pytorch → cuda, where `depends_on(cuda, driver)` succeeds. This proves the chain visionapp → pytorch → cuda → driver. The answer `true .` means a choice point was still open (for example the opencv branch), and we pressed `<enter>` instead of asking for more proofs.

```
?- trace.
true.

[trace] ?- requires(visionapp, driver).
   Call: (12) requires(visionapp, driver) ? creep
   Call: (13) depends_on(visionapp, driver) ? creep
   Fail: (13) depends_on(visionapp, driver) ? creep
   Redo: (12) requires(visionapp, driver) ? creep
   Call: (13) depends_on(visionapp, _17230) ? creep
   Exit: (13) depends_on(visionapp, pytorch) ? creep
   Call: (13) requires(pytorch, driver) ? creep
   Call: (14) depends_on(pytorch, driver) ? creep
   Fail: (14) depends_on(pytorch, driver) ? creep
   Redo: (13) requires(pytorch, driver) ? creep
   Call: (14) depends_on(pytorch, _22554) ? creep
   Exit: (14) depends_on(pytorch, cuda) ? creep
   Call: (14) requires(cuda, driver) ? creep
   Call: (15) depends_on(cuda, driver) ? creep
   Exit: (15) depends_on(cuda, driver) ? creep
   Exit: (14) requires(cuda, driver) ? creep
   Exit: (13) requires(pytorch, driver) ? creep
   Exit: (12) requires(visionapp, driver) ? creep
true .
```

Figure 4: Trace of the query `requires(visionapp, driver)`.

2.2 Clause order and cyclic dependencies

Both versions are in one file under different names, `requires_a` (Version A) and `requires_b` (Version B), so they run on the same facts (Figure 5).

```
% Exercise 2.2: Clause order and cyclic dependencies
% Queries: requires_a(visionapp, driver).
%          requires_b(visionapp, driver).

depends_on(visionapp, pytorch).
depends_on(visionapp, opencv).
depends_on(pytorch, cuda).
depends_on(opencv, numpy).
depends_on(cuda, driver).
% depends_on(driver, pytorch).   % uncomment for the cyclic dependency part

% Version A
requires_a(X, Y) :- depends_on(X, Y).
requires_a(X, Y) :- depends_on(X, Z), requires_a(Z, Y).

% Version B
requires_b(X, Y) :- depends_on(X, Z), requires_b(Z, Y).
requires_b(X, Y) :- depends_on(X, Y).
```

Figure 5: Knowledge base for Exercise 2.2 (`NanoGangstas_qn_2_2.pl`).

2.2.1 Effect of clause order

Clause order changes the proof search because Prolog tries a predicate's clauses from top to bottom. It searches depth-first: it goes as deep as it can along one path, and backs up to try another when that path fails. Figure 6 shows both traces side by side.

- **Version A** first checks whether driver is a direct dependency, and only goes one level deeper if it is not. It checks visionapp (no), pytorch (no), then cuda (yes). The proof takes 18 steps and never looks at driver's own dependencies.
- **Version B** goes deeper first, following visionapp → pytorch → cuda → driver. The package driver has no dependencies, so both clauses of `requires_b(driver, driver)` fail. Prolog backs up to `requires_b(cuda, driver)` and tries its second clause, which succeeds. This dead end (depth 15–16 in the trace) is why the proof takes 22 steps.

Version A (base clause first)

```
?- trace.
true.

[trace] ?- requires_a(visionapp, driver).
   Call: (12) requires_a(visionapp, driver) ? creep
   Call: (13) depends_on(visionapp, driver) ? creep
   Fail: (13) depends_on(visionapp, driver) ? creep
   Redo: (12) requires_a(visionapp, driver) ? creep
   Call: (13) depends_on(visionapp, _17266) ? creep
   Exit: (13) depends_on(visionapp, pytorch) ? creep
   Call: (13) requires_a(pytorch, driver) ? creep
   Call: (14) depends_on(pytorch, driver) ? creep
   Fail: (14) depends_on(pytorch, driver) ? creep
   Redo: (13) requires_a(pytorch, driver) ? creep
   Call: (14) depends_on(pytorch, _22590) ? creep
   Exit: (14) depends_on(pytorch, cuda) ? creep
   Call: (14) requires_a(cuda, driver) ? creep
   Call: (15) depends_on(cuda, driver) ? creep
   Exit: (15) depends_on(cuda, driver) ? creep
   Exit: (14) requires_a(cuda, driver) ? creep
   Exit: (13) requires_a(pytorch, driver) ? creep
   Exit: (12) requires_a(visionapp, driver) ? creep
true .
```

Version B (recursive clause first)

```
?- trace.
true.

[trace] ?- requires_b(visionapp, driver).
   Call: (12) requires_b(visionapp, driver) ? creep
   Call: (13) depends_on(visionapp, _14602) ? creep
   Exit: (13) depends_on(visionapp, pytorch) ? creep
   Call: (13) requires_b(pytorch, driver) ? creep
   Call: (14) depends_on(pytorch, _17262) ? creep
   Exit: (14) depends_on(pytorch, cuda) ? creep
   Call: (14) requires_b(cuda, driver) ? creep
   Call: (15) depends_on(cuda, _19922) ? creep
   Exit: (15) depends_on(cuda, driver) ? creep
   Call: (15) requires_b(driver, driver) ? creep
   Call: (16) depends_on(driver, _22582) ? creep
   Fail: (16) depends_on(driver, _22582) ? creep
   Redo: (15) requires_b(driver, driver) ? creep
   Call: (16) depends_on(driver, driver) ? creep
   Fail: (16) depends_on(driver, driver) ? creep
   Fail: (15) requires_b(driver, driver) ? creep
   Redo: (14) requires_b(cuda, driver) ? creep
   Call: (15) depends_on(cuda, driver) ? creep
   Exit: (15) depends_on(cuda, driver) ? creep
   Exit: (14) requires_b(cuda, driver) ? creep
   Exit: (13) requires_b(pytorch, driver) ? creep
   Exit: (12) requires_b(visionapp, driver) ? creep
true .
```

Figure 6: Traces of `requires(visionapp, driver)` under each clause order.

Both versions mean the same in logic, so they prove the same facts. Only the order of the proof steps changes, so the order of the answers changes too: Version A returns the direct dependencies first, and Version B returns the deepest first (Figure 7).

Version A

```
?- requires_a(visionapp, P).
P = pytorch ;
P = opencv ;
P = cuda ;
P = driver ;
P = numpy ;
false.
```

Version B

```
?- requires_b(visionapp, P).
P = driver ;
P = cuda ;
P = numpy ;
P = pytorch ;
P = opencv.
```

Figure 7: All answers to `requires(visionapp, P)` under each clause order.

2.2.2 Effect of a cyclic dependency

Adding the fact `depends_on(driver, pytorch)` creates a cycle: pytorch → cuda → driver → pytorch. The problem is that a query may never finish (**non-termination**). Prolog does not remember which goals it is already trying to prove, so it can go round this cycle forever. The results below are with the cycle fact uncommented.

Version A. `requires_a(visionapp, driver)` still gives its first answer, because the first clause finds driver at cuda, before the search gets back to pytorch. Asking for more answers goes round the cycle forever: the same answers repeat, and numpy is never reached. However, a query that should be false, such as `requires_a(cuda, opencv)`, never finishes and gives no error, so we stopped it with a time limit (Figure 8).

```
?- requires_a(visionapp, P).
P = pytorch ;
P = opencv ;
P = cuda ;
P = driver ;
P = pytorch ;
P = cuda ;
P = driver ;
P = pytorch ;
P = cuda ;
P = driver ;
P = pytorch ;
P = cuda ;
P = driver .

?- call_with_time_limit(10, requires_a(cuda, opencv)).
ERROR: Unhandled exception: Time limit exceeded
```

Figure 8: Version A on the cyclic knowledge base. Top: we stopped asking for more answers after 13; they keep repeating. Bottom: a query that should be false, stopped after 10 seconds.

Version B. Even the true query `requires_b(visionapp, driver)` gets no answer. The recursive clause is always tried first, so Prolog keeps following cuda → driver → pytorch → cuda → … and never tries the second clause. Each round leaves that untried clause waiting, so the stack keeps growing until SWI-Prolog raises a stack-overflow error. With the default 1 GB limit it was still running after 5 minutes, so we used `--stack-limit=64m`, which fails in seconds (Figure 9).

```
?- trace.
true.

[trace] ?- requires_b(visionapp, driver).
   Call: (12) requires_b(visionapp, driver) ? creep
   Call: (13) depends_on(visionapp, _14602) ? creep
   Exit: (13) depends_on(visionapp, pytorch) ? creep
   Call: (13) requires_b(pytorch, driver) ? creep
   Call: (14) depends_on(pytorch, _17262) ? creep
   Exit: (14) depends_on(pytorch, cuda) ? creep
   Call: (14) requires_b(cuda, driver) ? creep
   Call: (15) depends_on(cuda, _19922) ? creep
   Exit: (15) depends_on(cuda, driver) ? creep
   Call: (15) requires_b(driver, driver) ? creep
   Call: (16) depends_on(driver, _22582) ? creep
   Exit: (16) depends_on(driver, pytorch) ? creep
   Call: (16) requires_b(pytorch, driver) ? creep
   Call: (17) depends_on(pytorch, _25242) ? creep
   Exit: (17) depends_on(pytorch, cuda) ? creep
   Call: (17) requires_b(cuda, driver) ? creep
   Call: (18) depends_on(cuda, _27902) ? creep
   Exit: (18) depends_on(cuda, driver) ? creep
   Call: (18) requires_b(driver, driver) ? creep
   Call: (19) depends_on(driver, _30562) ? creep
   Exit: (19) depends_on(driver, pytorch) ? creep
   Call: (19) requires_b(pytorch, driver) ? creep
   Call: (20) depends_on(pytorch, _33222) ? creep
   Exit: (20) depends_on(pytorch, cuda) ? creep
   Call: (20) requires_b(cuda, driver) ? abort

?- requires_b(visionapp, driver).
ERROR: Stack limit (64.0Mb) exceeded
ERROR:   Stack sizes: local: 45.3Mb, global: 2.3Mb, trail: 0Kb
ERROR:   Stack depth: 296,753, last-call: 0%, Choice points: 296,746
ERROR:   Possible non-terminating recursion:
ERROR:     [296,752] user:requires_b(pytorch, driver)
ERROR:     [296,751] user:requires_b(driver, driver)
   Exception: (296,752) requires_b(pytorch, driver) ? abort
```

Figure 9: Version B on the cyclic knowledge base. Top: the trace, aborted once the repeating pattern was clear. Bottom: the same query without tracing.

The cause is the cycle together with Prolog's depth-first search, which does not check for loops. Changing the clause order only decides whether an answer happens to be found before the search starts looping. Neither order removes the loop.